

Question 1

Your agile and ever-creative friend, Stella proposes an interesting app idea to you that, if implemented successfully, will allow her to reach her fitness goals.

Everyday Stella runs for T seconds. In order to maintain a suitable running speed she wants to create a Spotify playlist of total length T where each song, s_i has length $time_i$ and bpm, bpm_i (beats per minute). Her aim is to run in time to the bpm of each song, however, maintaining a uniform bpm for each session will not efficiently scale Stella's running speed over time. Instead, she wishes for the bpm of successive songs to approximate a cubic polynomial, $P(x)$.

Stella provides you with a large song bank, U of n unique songs and a cubic, $P(x)$. A playlist, S is an ordered list of songs where each $s_i \in U$ and the sum of all song lengths is T . A playlist can have multiple instances of the same song. For each $s_i \in S$ let there be an (x_i, y_i) where $x_0 = 0$ and $x_i = x_{i-1} + time_i$ and $y_i = bpm_i$. Let the error for a song be α_i where α_i is the area between the graph $y = y_i$ and $y = P(x)$ in the range $[x_{i-1}, x_i]$. Let the total error for a playlist be $\sum_1^{|S|} \alpha_i$.

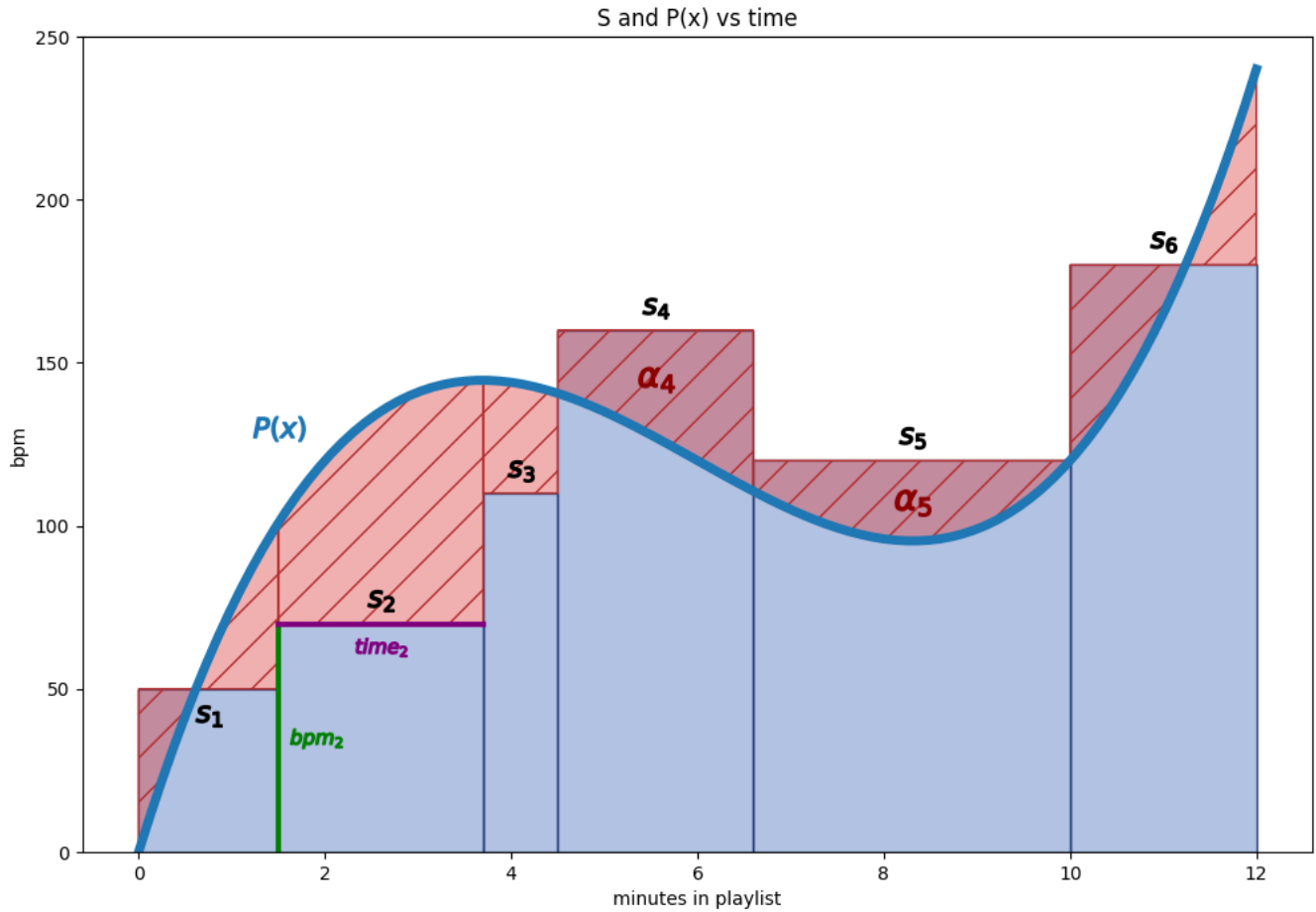


Figure 1: $P(x)$ and a possible S vs time

1.1 By designing an algorithm or otherwise, show that the error of a song can be computed in $O(1)$.

The error of a song s_i is the area between $y = y_i$ and $P(x)$ in the range $[x_{i-1}, x_i]$. Let us assume that $y = y_i$ does not intersect $P(x)$ in the range $[x_{i-1}, x_i]$. Then α_i is simply $|\int_{x_{i-1}}^{x_i} (P(x) - y_i) dx|$. Since a cubic has a maximum of 4 terms then, iterating through $P(x)$ to construct the definite integral, $P'(x)$ will take constant time. α_i will then be in the form $\alpha_i = |\int_{x_{i-1}}^{x_i} (P(x) - y_i) dx| = [P'(x_i) - y_i x_i] - [P'(x_{i-1}) - y_i x_{i-1}]$. Since $P'(x)$ will have a maximum of 4 terms then the above calculation is $O(1)$.

Now if $y = y_i$ intersects $P(x)$ then it will have a maximum of 3 intersections, as $P(x) - y_i = 0$ will have a maximum of 3 real solutions. Our algorithm can use the *Cubic Formula* to find these intersections. If we order these intersections by x value, let them be x_j, x_k, x_l , then we can calculate the error like such. $|\int_{x_{i-1}}^{x_j} (P(x) - y_i) dx| + |\int_{x_j}^{x_k} (P(x) - y_i) dx| + |\int_{x_k}^{x_l} (P(x) - y_i) dx| + |\int_{x_l}^{x_i} (P(x) - y_i) dx|$. Songs with intersections less than 3 will have a similar form but with strictly fewer terms. Since we have shown that calculating one of these integrals is $O(1)$, then this entire calculation must be $O(1)$.

[2 marks]

- [1 mark] Shows that finding the integral requires constant calculations
- [1 mark] Shows that intersections will have a constant upper bound

1.2 Design an algorithm, in English, that finds the playlist of minimal total error in $O(Tn)$. Assume there exists a playlist of length T .

This problem is analogous to the *Canoe Problem* from *Module 3 Live Lecture Exercises - W5 - Question 1*.

Subproblem(k) : What is the minimal total error for a playlist of length k?

A playlist must have a final song. We can appropriate the algorithm from **1.1** to find the error of this song. Let this final song be $u_j \in U$ and let $error(k - time_j, k, bpm_j)$ be the area between $y = bpm_j$ and $P(x)$ in the range $[k - time_j, k]$. This will be the error of a song u_j at the end of a playlist of length k .

If we include the song u_j as the last song in our playlist, then we know the minimal error must be the minimal error required to reach the time when the final song starts plus the error of the final song. This is the same as $opt(k - time_j) + error(k - time_j, k, bpm_j)$. If we calculate this value for all songs, one of these values will be a minimum. That song will be the most optimal choice to end a playlist of size k on. This recurrence is shown below:

$$\begin{aligned} opt(0) &= 0 \\ opt(k) &= \min_{1 \leq j \leq n} (opt(k - time_j) + error(k - time_j, k, bpm_j)) \quad \text{if } k > 0 \end{aligned}$$

Consider the case where a playlist only has 1 song. Then we want the error for this playlist to be equal to the error of its only song. If we assume this song produces the minimal error

for $k = \text{time}_j$ then

$$\begin{aligned} \text{opt}(\text{time}_j) &= \text{opt}(\text{time}_j - \text{time}_j) + \text{error}(\text{time}_j - \text{time}_j, \text{time}_j, \text{bpm}_j) \\ &= \text{opt}(0) + \text{error}(0, \text{time}_j, \text{bpm}_j) \end{aligned}$$

But the error of this playlist must be equal to the error of its only song. Therefore $\text{opt}(0)$ must be 0. This is how we derive our base case $\text{opt}(0) = 0$.

We must also consider the case where a song exceeds the length of a playlist. We cannot include this song in the playlist, so if $\text{time}_j > k$ then we set $\text{error}(k - \text{time}_j, k, \text{bpm}_j) = \infty$. If we consider a k where k is less than the shortest song duration in U , then all songs for $\text{opt}(k)$ will have $\text{time}_j > k$, resulting in $\text{opt}(k) = \infty$. So certain playlists will have no solution. Fortunately the question assumes that some playlist of length T exists, so $\text{opt}(T)$ must produce a value.

To find $\text{opt}(T)$, we must construct an array of size T and calculate $\text{opt}(k)$ for $1 \leq k \leq T$. To find the songs included in the playlist define the following recurrence.

$$\text{optseq}(k) = \underset{1 \leq j \leq n}{\text{argmin}}(\text{opt}(k - \text{time}_j) + \text{error}(k - \text{time}_j, k, \text{bpm}_j))$$

After we find $\text{opt}(T)$ we can recursively backtrack from T to 0 to produce song indices. If we find $\text{optseq}(T) = j$ then j is the song index of the of the optimal song to end a playlist of length T on. If we then find $\text{optseq}(T - \text{time}_j)$ then we can find the second last song index. If we continue this process until we reach index 0, then we can then substitute all indices into U to find the optimal playlist. Since we have to calculate $\text{opt}(k)$ T times and each calculation requires finding errors for n songs taking $O(1)$ time each, then the algorithm is $O(Tn)$.

[8 marks]

- [2 marks] States the sub-problem
- [2 marks] States and explains recurrence equations
- [1 marks] States base case
- [1 marks] Accounts for edge case where a song exceeds the length of a playlist
- [2 marks] Justifies time complexity

1.3 Stella really appreciates your efforts, but she finds that your algorithm has an unfortunate taste for the incoherent. Unlike your entropy obsessed algorithm, Stella believes that the perfect playlist can be partitioned into 2 distinct sections, where all songs from a section belong to a genre of Stella's choosing. Stella also chooses the order in which genres appear in the playlist. The playlist can be partitioned unevenly, however each section must contain at least one song from its corresponding genre. Each song in U belongs to exactly one genre.

Design an algorithm, in English, that finds the playlist of minimal total error in $O(Tn)$ such that the playlist adheres to Stella's new guidelines. Assume there exists a playlist of length T that matches the requirements.

If a playlist is partitioned at time p , then we want to find the playlist of minimal total error that starts at time 0 and ends at time p that has all songs belonging to $genre_1$ and we want to find the playlist of minimal total error that starts at time p and ends at time T that has all songs belonging to $genre_2$. If we concatenate these playlists we will have a playlist of length T and the sum of their errors will be the minimal total error of a playlist with partition at time p . Since the playlist does not have to be partitioned evenly, we must find the error of all playlists partitioned at p for all $1 \leq p \leq T$. One of these errors will be a minimum. This will be the optimal playlist.

Let $opt'(T)$ be this minimal value. Then the above can be expressed as follows

$$opt'(T) = \min_{1 \leq p \leq T} (opt'_1(0, p) + opt'_2(p, T))$$

where $opt'_g(a, b)$ is the playlist of total minimal error starting at time a and ending at time b only containing songs from genre g .

To find $opt'_1(0, p)$ for all $1 \leq p \leq T$ we simply precalculate $opt(k)$ from **1.2** for all values $1 \leq k \leq T$, instead only considering songs belonging to $genre_1$ from U . This precalculation will take $O(Tn)$ and after finding $opt(T)$ then finding any $opt'_1(0, p)$ will take constant time. We can find $opt'_2(p, T)$ with a similar method, however even after precalculations we will not be able to find $opt'_2(p, T)$ in constant time as in contrast to $opt'_1(0, p)$, the end time of the playlist is now fixed and the start time of the playlist becomes a variable amount. Instead consider the following subproblem.

Subproblem(m, T): What is the minimal total error for a playlist starting at time m and ending at time T ?

To solve this problem, we can take the approach from **1.2** but in reverse. A playlist must start with a song. Let this first song be u_j and let $error'(m, m + time_j, bpm_j)$ be the area between $y = bpm_j$ and $P(x)$ in the range $[m, m + time_j]$. This will be the error of a song u_j at the start of a playlist beginning at time m . Only songs that end before time T can be added to this playlist so let $error'(m, m + time_j, bpm_j) = \infty$ if $m + time_j > T$. If we begin a playlist with song u_j then it is clear that the minimal error for this playlist will be the error of this first song plus the minimal error it takes to reach T from the time the first song ends, $m + time_j$. This is the same as $error'(m, m + time_j, bpm_j) + opt'_2(m + time_j, T)$. If we calculate this error for all songs in U that belong to $genre_2$ then one of them will be a minimum. The song that results in the minimal error will be the optimal selection for the first song of a playlist that starts at time m . This recurrence is as follows.

$$\begin{aligned} opt'_2(T, T) &= 0 \\ opt'_2(m, T) &= \min_{\substack{1 \leq j \leq n \\ u_j \in genre_2}} (error'(m, m + time_j, bpm_j) + opt'_2(m + time_j, T)) \quad \text{if } m < T \end{aligned}$$

To find $opt'_2(0, T)$ construct an array of size T and calculate $opt'_2(m, T)$ for all $0 \leq m < T$ beginning at time $m = T - 1$ and decreasing to $m = 0$. Since we have precalculated all $opt'_2(p, T)$ in the range $0 \leq p \leq T$ then finding any $opt'_2(p, T)$ will take constant time. Thus, finding $opt'(T)$ will require $T O(1)$ comparisons. Since precalculations are $O(Tn)$ then in total the algorithm is $O(Tn)$.

[8 marks]

- [2 marks] States new subproblem

- [2 marks] States base case
- [2 marks] Explains and states recurrence equation(s)
- [1 marks] Algorithm is $O(Tn)$
- [1 marks] Justifies time complexity

Group number: Goat Yoghurt

Name	zID	Contributions
Ashley Clayton	z5406283	1/4
Mark Raya	z5424285	1/4
James Zheng	z5417110	1/4
Julian Britton	z5309173	1/4

Question inspired by a question asked by my actual friend Stella Mountain.

Solution inspired by the *Canoe Problem* from *Module 3 Live Lecture Exercises W5 Question1*